Non-Invasive Anemia Screening Data Pipeline & CNN Benchmarking Suite

Project Scope: B.Tech Term Project — Work Package A (Data Pipeline & Benchmarking)

Target Biomarker: Palpebral Conjunctiva (Eye Mucosa)

Primary Dataset: CP-AnemiC (Ghana) & Clinical Metadata

Stack: PyTorch, OpenCV, Scikit-Learn, Pandas

1. Executive Summary & Clinical Problem Formulation

Anemia is a major public-health problem in India and developing regions, especially in rural areas where invasive venous blood testing is costly, slow, and constrained by logistical hurdles. This project develops an automated, reproducible deep learning system for **non-invasive anemia screening** using smartphone photographs of the **eye conjunctiva** (the vascularized lower palpebral mucosal membrane). A central challenge is that images are captured across diverse smartphone sensors and uncontrolled lighting conditions.

In direct fulfillment of **Work Package A** specifications, this software provides:

- **Dataset Loader & Clinical Audit Engine:** Parses and cross-references CP-AnemiC images against clinical records (hemoglobin levels, age, gender, severity), isolating mismatches in an audit log rather than silently dropping or trusting corrupted samples.
- **Illumination-Robust ROI Extraction:** Multi-color space segmentation (CIELAB, YCrCb, HSV, Norm-RGB) with morphological horizontal crescent bridging, anatomical position scoring, and automatic center-crop safety fallback.
- **Group-Aware K-Fold CNN Training Harness:** Patient-level stratified cross-validation benchmarking transfer learning CNNs (ResNet-50 and EfficientNet-B0) with inverse class weighting, Cosine Annealing, and crash-resilient checkpointing.
- **Clinical Decision Threshold Optimization:** Zero-retraining post-hoc probability calibration across out-of-fold predictions to enforce diagnostic safety constraints (≥ 90% sensitivity).
- **Multi-Model Benchmarking Suite:** Automated hyperparameter sweeping comparing distinct CNN backbones, learning rates, weight decay, and random erasing regularization.

2. End-to-End Pipeline Architecture & Workflow

The system is managed via a single configuration file (`configs/config.yaml`), guaranteeing complete parameter traceability and deterministic reproducibility.

Stage	Primary Module / Script	Algorithmic Operations & Methods	Outputs & Artifacts
Stage 1: Ingestion & Audit	`src/dataset_loader.py` `scripts/run_pipeline.py`	Regex ID normalization, primary key relational join with Excel sheet, folder-vs-clinical label verification.	`cp_anemic_metadata.csv` `cp_anemic_issues.csv`

Stage 2: Conjunctiva ROI	`src/roi_extraction.py`
	`scripts/test_roi_on_folder.py`
		Multi-space chromaticity analysis (Lab, YCrCb, HSV, Norm-RGB), Mucosal Redness Index (MRI), crescent morphology, geometric scoring.	`outputs/roi/ (Crops)`
			`outputs/debug/ (QA)`
			`cp_anemic_metadata_with_roi.csv`

Stage 3: CNN Training	`src/train.py`
	`src/dataset.py`
	`src/model.py`
		StratifiedGroupKFold on `patient_id`, inverse class weighting, Cosine Annealing, AMP mixed precision, crash-resilient checkpoints.	`fold_*/best_model.pt`
			`baseline_results_table.csv`
			Confusion Matrix / ROC / PR plots

Stage 4: Threshold Tuning	`src/threshold_analysis.py`
	`scripts/analyze_thresholds.py`
		Zero-retraining Out-of-Fold (OOF) inference pooling, threshold sweep (0.05–0.95), Youden's J & Sensitivity Floor optimization.	`oof_predictions.csv`
			`threshold_sweep.png`
			`confusion_matrix_recommended.png`

Stage 5: Experiment Suite	`src/experiment_runner.py` `scripts/run_experiments.py`	Systematic architectural comparison (ResNet-50 vs EfficientNet-B0), learning rate & regularization sweeps.	`outputs/experiments/` `leaderboard.csv`

3. Technical Deep-Dive: File-by-File Codebase Walkthrough

Stage 1: Clinical Dataset Loader & Integrity Auditing

Source File: `src/dataset_loader.py` (Invoked via `scripts/run_pipeline.py --stage metadata`)

- **Image Directory Scanning:** Scans `Anemic` and `Non-anemic` directories for valid image formats (`.jpg`, `.png`, `.jpeg`, `.bmp`).
- **Fuzzy ID Normalization:** Function `_normalize_id()` strips punctuation, whitespace, and casing using regex (`re.sub(r"[^a-z0-9]", "", raw.lower())`) so variations like `Image_001.jpg`, `image001.png`, and `IMAGE-001.jpeg` map to the same key `image001`.
- **Relational Joining with Clinical Metadata:** Matches normalized IDs against `Anemia_Data_Collection_Sheet.xlsx`, parsing quantitative hemoglobin (`HB_LEVEL` in g/dL), severity (`Mild`, `Moderate`, `Severe`), patient age, gender, hospital, and geographic region.
- **Audit Quarantining:** Compares on-disk folder labels against the clinical `REMARK` column. Confirmed entries export to `cp_anemic_metadata.csv`. Any anomalies (label disagreements, missing files, or unindexed rows) are isolated in `cp_anemic_issues.csv` with explicit diagnostic tags.

Stage 2: Illumination-Robust Conjunctiva ROI Extraction

Source File: `src/roi_extraction.py` (Invoked via `scripts/run_pipeline.py --stage roi`)

Palpebral conjunctiva tissue is characterized by microvascular perfusion that produces localized erythema. To segment this tissue robustly across varying lighting and skin tones, the module uses a 5-step computer vision algorithm:

1. **Multi-Space Color Transformations:** CIELAB (L^*, a^*, b^*) isolates vascular redness (a^*) and subtracts yellowness (b^*); YCrCb isolates red chroma (Cr) independent of luminance; Normalized Excess Red computes $(R - G) / (R + G + B + \varepsilon)$; HSV isolates red hue angles ($H \le 26° \cup H \ge 152°$, $S \ge 28$).
2. **Composite Mucosal Redness Index (MRI):** Formulates a continuous redness metric reinforcing mucosal tissue:
   $$\text{MRI} = (a^* - 128) + (Cr - 128) + 80 \cdot (r_{norm} - g_{norm}) + 0.15 \cdot S - 0.25 \cdot (b^* - 128)$$
3. **Eye-Zone Adaptive Thresholding:** Rejects flash glare ($V > 245$, $S < 35$) and dark shadows ($V < 40$). Computes 95th and 68th percentiles of MRI in the ocular window to dynamically set a cutoff.
4. **Horizontal Crescent Morphological Bridging:** Applies an elliptical structuring element ($k_w \approx 4.5\%$ width, $k_h \approx 1.5\%$ height) via morphological closing to bridge palpebral vascular arches.
5. **Connected Component Anatomical Scoring:** Evaluates detected blobs using an anatomical fitness function:
   $$\text{Score} = \text{Area} \cdot \text{PositionWeight}(y_{rel}) \cdot \text{AspectWeight}(w/h) \cdot (\text{Mean\_MRI})^{1.3}$$
6. **Safety Bounding Box & Fallback Crop:** Adds safety padding (8% vertical, 4% horizontal). If segmentation fails, automatically defaults to a central 60% window (`status: "fallback"`). Multiprocessing with `ProcessPoolExecutor` accelerates batch execution.

Stage 3: PyTorch Dataset & Clinical Augmentation

Source File: `src/dataset.py`

- **`ConjunctivaDataset`:** Lazy-loads images via OpenCV, handles color conversions, applies transforms, and maps labels (`0: Non-anemic`, `1: Anemic`).
- **Domain-Appropriate Augmentation:** Applies horizontal flipping ($p = 0.5$), mild rotation (±15° to ±20°), and color jitter (brightness, contrast, saturation).
- **Preservation of Anatomical Priors:** *No vertical flips are permitted* because ocular images possess strict vertical orientation (sclera above, lower eyelid below).
- **Random Erasing Regularization:** Optionally masks a random rectangular sub-patch (2%–15% area) with probability $p = 0.25$, forcing the network to evaluate broader mucosal tissue rather than overfitting to single vascular clusters.

Stage 4: CNN Architecture Factory

Source File: `src/model.py`

- `build_model()` instantiates standard convolutional architectures with ImageNet-pretrained weights and adapts their final classification heads:
  - **ResNet-50:** Loads `ResNet50_Weights.IMAGENET1K_V2` and replaces the final layer: `model.fc = nn.Linear(2048, 2)`.
  - **EfficientNet-B0:** Loads `EfficientNet_B0_Weights.IMAGENET1K_V1` and replaces the classifier: `model.classifier[1] = nn.Linear(1280, 2)`.
- **Backbone Freezing:** Supports `freeze_backbone=True` (feature extraction where only the head receives gradients) or `freeze_backbone=False` (end-to-end full fine-tuning).
- `count_trainable_params()` audits model parameter allocation prior to training.

Stage 5: Cross-Validation Training & Crash-Recovery Harness

Source File: `src/train.py` (Invoked via `scripts/train_baseline.py`)

- **Patient-Level Stratified K-Fold:** Uses `StratifiedGroupKFold(n_splits=5)` grouped on `patient_id`. Guarantees that all images from the same subject remain strictly within either training or validation, preventing data leakage. An explicit runtime assertion (`assert not overlap`) verifies split purity.
- **Class Imbalance Reweighting:** Addresses the 424 : 286 (60% : 40%) anemic to non-anemic class ratio by computing inverse class frequencies: $w_c = N / (C \cdot N_c)$ and injecting them into `nn.CrossEntropyLoss(weight=class_weights)`.
- **Mixed Precision Acceleration:** Enables CUDA Automatic Mixed Precision (AMP via `torch.amp.autocast` and `GradScaler`), halving memory consumption and accelerating training.
- **Learning Rate Scheduling:** Supports **Cosine Annealing (`CosineAnnealingLR`)** for smooth learning rate decay, or **ReduceLROnPlateau**.
- **Early Stopping on F_2 Metric:** Monitors validation F_2 score each epoch with patience (10 epochs), immediately saving `best_model.pt` whenever a new peak is achieved.
- **Crash-Resilient State Checkpointing:** Saves `resume_state.pt` every epoch (model weights, optimizer, scaler, scheduler, history, patience counters). If interrupted, re-running skips completed folds and resumes unfinished folds from the exact interrupted epoch.

Stage 6: Statistical Metrics & Publication Figures

Source File: `src/metrics.py`

- Computes **Accuracy, Sensitivity (Recall on Anemic), Specificity, F_1, F_2, and ROC AUC**.
- Generates high-resolution (300 DPI) publication-ready plots: `confusion_matrix.png`, `roc_curve.png`, `pr_curve.png`, and `training_history.png`.
- Aggregates fold metrics into `baseline_results_table.csv` with mean and standard deviation rows.

Stage 7: Post-Hoc Clinical Decision Threshold Analysis

Source File: `src/threshold_analysis.py` (Invoked via `scripts/analyze_thresholds.py`)

- **Zero-Retraining OOF Inference:** Loads trained fold checkpoints (`best_model.pt`), evaluates held-out validation images, and pools predictions into a single Out-of-Fold (OOF) dataset (`oof_predictions.csv`).
- **Threshold Sweeping:** Evaluates candidate cutoffs $\tau \in [0.05, 0.95]$ in increments of 0.01.
- **Three Clinical Recommendations:** (1) `max_f2`: Maximizes F_2 score directly; (2) `max_youden_j`: Maximizes Youden's Index (J = Sensitivity + Specificity − 1); (3) `max_specificity_at_sensitivity_floor`: Enforces Sensitivity ≥ 90% and maximizes specificity.
- Outputs `threshold_sweep.png` and `confusion_matrix_recommended_threshold.png`.

Stage 8: Multi-Model Experimentation & Benchmark Suite

Source File: `src/experiment_runner.py` (Invoked via `scripts/run_experiments.py`)

- Automates sequential benchmarking across all model architectures and hyperparameter overrides defined in `config.yaml`.
- Isolates outputs per experiment (`outputs/experiments/<run_name>/`) and exports a ranked comparative `leaderboard.csv`.

4. CNN Architectures & Experimental Variations Breakdown

ResNet-50 Architecture

- **Total Parameters:** ~25.6 Million
- **Building Blocks:** 3-layer Bottleneck Residual Units (1×1 → 3×3 → 1×1).
- **Mechanism:** Identity shortcut connections eliminate vanishing gradients in deep networks.
- **Clinical Role:** High-capacity baseline capable of modeling subtle vascular features.

EfficientNet-B0 Architecture

- **Total Parameters:** ~5.3 Million (5x lighter than ResNet-50)
- **Building Blocks:** MBConv (Inverted Residuals) + Squeeze-and-Excitation (SE).
- **Mechanism:** Compound scaling balances depth, width, and resolution; SE provides channel attention.
- **Clinical Role:** Parameter-efficient architecture that resists overfitting on compact datasets.

The 6 Experimental Variations Evaluated

1. `resnet50_lr1e-4`: ResNet-50 baseline with Adam optimizer, LR = 1×10^{-4}, Cosine Annealing, and standard data augmentation.
2. `resnet50_lr3e-5`: ResNet-50 with conservative learning rate (LR = 3×10^{-5}) to evaluate weight preservation.
3. `efficientnet_b0_lr1e-4`: EfficientNet-B0 baseline with LR = 1×10^{-4} and Cosine Annealing.
4. `efficientnet_b0_lr3e-4`: EfficientNet-B0 with elevated learning rate (LR = 3×10^{-4}) to test convergence speed.
5. `efficientnet_b0_lr1e-4_wd5e-4`: EfficientNet-B0 with 5× increased L2 weight decay (5×10^{-4}) to penalize large weights.

6. `efficientnet_b0_lr1e-4_wd5e-4_aug`: EfficientNet-B0 combining weight decay (5×10^{-4}), enhanced rotation (±20°), color jitter (0.3), and **Random Erasing** (p = 0.25).

Empirical Benchmark Results (5-Fold Stratified Group Cross-Validation)

The table below presents the official benchmark results aggregated across all 5 cross-validation folds from `outputs/experiments/leaderboard.csv`:

Experiment Configuration	Mean Accuracy	Mean Sensitivity	Mean Specificity	Mean F1	Mean F2	Mean ROC AUC	Key Empirical Observations
`resnet50_lr1e-4`	84.23% ± 3.1%	**91.90% ± 4.4%**	72.68% ± 5.5%	0.8742	**0.9003 ± 0.033**	0.9209 ± 0.023	**Highest F2 Score (0.9003)** and top recall (91.9%). Strongest pure screening baseline.
`efficientnet_b0_lr1e-4_wd5e-4_aug`	**86.06% ± 3.6%**	90.39% ± 6.4%	80.71% ± 7.6%	0.8845	0.8954 ± 0.047	0.9207 ± 0.022	**Best Clinical Balance:** Exceeds 90% sensitivity while elevating specificity to 80.7%.
`efficientnet_b0_lr1e-4`	**86.62% ± 2.4%**	88.49% ± 3.4%	**84.25% ± 5.4%**	**0.8866**	0.8854 ± 0.027	**0.9277 ± 0.021**	**Highest ROC AUC (0.928)** and lowest fold variance (±2.4%), proving superior generalizability.
`efficientnet_b0_lr3e-4`	81.55% ± 9.2%	92.02% ± 5.6%	65.86% ± 18.2%	0.8565	0.8933 ± 0.059	0.8644 ± 0.105	High sensitivity but unstable fold variance (±18% specificity) due to high learning rate.
`resnet50_lr3e-5`	81.69% ± 2.8%	89.17% ± 5.0%	71.30% ± 5.2%	0.8525	0.8751 ± 0.034	0.8872 ± 0.020	Underperformed 1e-4; learning rate was too small for optimal convergence.
`efficientnet_b0_lr1e-4_wd5e-4`	84.51% ± 4.8%	87.51% ± 4.7%	79.43% ± 8.5%	0.8708	0.8733 ± 0.042	0.8988 ± 0.071	Solid balance; outperformed when enhanced augmentations were combined.

5. Professor Oral Examination & Defense Q&A Guide

Q1: Why did you designate the F2 score as the primary optimization metric rather than Accuracy or F1?

Answer: In non-invasive medical screening, clinical error costs are fundamentally asymmetric. A **False Negative** (failing to identify an anemic patient) leaves a patient untreated, whereas a **False Positive** merely results in a confirmatory venous blood test. The F_β metric with β = 2 weights Recall (Sensitivity) twice as heavily as Precision. This ensures that early stopping and checkpoint selection actively penalize missed anemic cases.

Q2: Why was StratifiedGroupKFold on patient_id required instead of standard StratifiedKFold?

Answer: If multiple images exist for the same subject, standard random splitting can place Image A of Patient #1 in the training set and Image B of Patient #1 in the validation set. The neural network would memorize patient-specific skin tone, lighting, and ocular geometry (data leakage) rather than generalizable anemia markers. Group K-Fold strictly guarantees that all images of any subject remain isolated within a single split.

Q3: Why develop a Multi-Space Chromaticity algorithm for ROI extraction instead of simple RGB or HSV thresholding?

Answer: Palpebral conjunctiva tissue and periocular eyelid skin share overlapping RGB and Hue distributions. Our **Composite Mucosal Redness Index (MRI)** combines CIELAB a^* (perfusion erythema), YCrCb Cr (red chroma), and normalized $R-G$, while actively subtracting b^* (epidermal yellowness). Combined with horizontal crescent morphological closing and spatial position scoring, this isolates vascularized conjunctiva tissue regardless of skin tone or flash lighting.

Q4: What is the technical distinction between Loss Reweighting and Decision Threshold Tuning?

Answer: They operate on two completely different stages of the learning process:

1. **Loss Reweighting (`use_class_weights`):** Modifies the gradient loss surface during backpropagation, forcing the network to penalize minority class errors during training.
2. **Threshold Tuning (`analyze_thresholds.py`):** Adjusts the post-hoc decision cutoff τ on out-of-fold probability outputs without retraining, allowing us to enforce clinical safety floors (e.g., Sensitivity ≥ 90%) or maximize Youden's J-statistic.

6. Command-Line Reproduction Guide

```
# 1. Execute Data Ingestion (Stage 1) & Conjunctiva ROI Extraction (Stage 2)
python scripts/run_pipeline.py

# 2. Execute Baseline 5-Fold Cross-Validation Training
python scripts/train_baseline.py

# 3. Perform Post-Hoc Decision Threshold Analysis on Out-of-Fold Probabilities
python scripts/analyze_thresholds.py --sensitivity-floor 0.90

# 4. Run the Full Multi-Model Benchmark Suite (Generates leaderboard.csv)
python scripts/run_experiments.py

# 5. Test ROI Extraction on any Arbitrary External Folder of Uncropped Photos
python scripts/test_roi_on_folder.py --input path/to/images --output outputs/roi_test
```

> **Summary & Next Research Steps**
>
> This pipeline delivers the full software deliverables specified in Work Package A. The modular design enables direct extension to external cross-dataset generalization testing (e.g., Indian clinical cohorts) and seamless integration with the Work Package B smartphone image capture application.